

INSTRUCTIONS PROJET - Système de Conseillers Littéraires Multi-Agents

OBJECTIF PRINCIPAL

Créer un système complet de recommandation de livres avec deux agents spécialisés :

- **Agent Tech** : Recommandations livres informatique/programmation/data science
- **Agent Littéraire** : Recommandations littérature générale

CONTEXTE ET CONTRAINTES

Technologies Confirmées

- **Backend** : Django 4.2.7 (déjà dans requirements.txt)
- **Agents LLM** : `google/flan-t5-base` (modèle léger mais cultivé)
- **RAG** : ChromaDB + `sentence-transformers/all-MiniLM-L6-v2`
- **API** : FastAPI pour les agents
- **Base de données** : PostgreSQL

Datasets Disponibles (dans `rags/*/data/`)

- **Agent Tech** : CSV dans `rags/tech_rag/data/` avec colonnes (title, description, author, rating, price, etc.)
- **Agent Littéraire** : 2 CSV dans `rags/literature_rag/data/` à fusionner (dataset complet + scores validation)

ARCHITECTURE ACTUELLE CONFIRMÉE

dual-book-advisor/	#  Racine projet
├── agents/	#  Agents LLM
│ └── config/	# Configuration agents
│ └── prompts/	# Templates prompts
├── api/	#  API FastAPI
│ └── middleware/	# Middleware API
│ └── models/	# Modèles Pydantic
│ └── routes/	# Routes API
│ └── services/	# Logique métier API
│ └── utils/	# Utilitaires API
├── apps/	#  Applications Django (à créer)
├── config/	#  Configuration projet
│ └── settings/	# Settings Django par environnement
├── rags/	#  Système RAG
│ └── tech_rag/	# RAG livres techniques
│ │ └── data/	#  CSV tech books ici
│ └── literature_rag/	# RAG littérature générale
│ │ └── data/	#  CSV littérature ici
│ └── shared/	# Utilitaires RAG partagés
├── scripts/	#  Scripts utilitaires
├── tests/	#  Tests
│ └── test_agents/	
│ └── test_api/	
│ └── test_rags/	
│ └── test_web/	
└── venv/	#  Environnement virtuel

PERSONNALITÉS DES AGENTS

Agent Tech

- **Style** : Pragmatique, orienté solutions
- **Exemples** : "Pour React, je recommande 'Fullstack React' (4.6  , \$39) - projets complets, très à jour avec React 18"
- **Logique** : Adapte selon niveau utilisateur (débutant/expert)

Agent Littéraire

- **Style** : Cultivé, découvreur, empathique
- **Exemples** : "Ah, Camus ! Tu pourrais adorer 'La Nausée' de Sartre - cette exploration du vide existentiel..."
- **Logique** : Mix découvertes/zone de confort selon profil

TÂCHES PRIORITAIRES

Phase 1 : Fondations (URGENT)

1. Setup Django dans apps/

bash

Créer les applications Django dans apps/

cd apps/

django-admin startapp accounts

django-admin startapp books

django-admin startapp chat

django-admin startapp dashboard

2. Configuration Django

- **config/settings/base.py** : Configuration PostgreSQL, apps Django
- **config/settings/development.py** : Settings dev
- **config/settings/production.py** : Settings prod
- **config/urls.py** : URLs racine
- **config/wsgi.py** et **config/asgi.py** : Serveur

3. Modèles Django Prioritaires

- **apps/accounts/models.py** : User, UserProfile avec préférences
- **apps/books/models.py** : TechBook, LiteratureBook, Recommendation
- **apps/chat/models.py** : ConversationHistory, Message

Phase 2 : Système RAG

4. RAG Tech Books

- **rags/tech_rag/data/** : Tes CSV tech books (déjà présents)
- **rags/tech_rag/tech_rag_manager.py** : Gestionnaire RAG tech
- **rags/tech_rag/processors/** : Scripts traitement CSV
- **rags/tech_rag/chroma_db/** : Base vectorielle ChromaDB

5. RAG Literature Books

- **rags/literature_rag/data/** : Tes 2 CSV littérature (déjà présents)
- **rags/literature_rag/literature_rag_manager.py** : Gestionnaire RAG littérature
- **rags/literature_rag/processors/** : Scripts fusion CSV + traitement
- **rags/literature_rag/chroma_db/** : Base vectorielle ChromaDB

6. RAG Partagé

- **rag/shared/embedding_manager.py** : Gestionnaire embeddings sentence-transformers
- **rag/shared/vector_store.py** : Interface ChromaDB commune
- **rag/shared/similarity_search.py** : Logique recherche vectorielle

Phase 3 : Agents LLM

7. Structure Agents

- **agents/base_agent.py** : Classe de base pour agents
- **agents/tech_agent.py** : Agent spécialisé tech avec flan-t5-base
- **agents/literature_agent.py** : Agent littérature avec flan-t5-base
- **agents/config/model_config.py** : Configuration flan-t5-base
- **agents/prompts/tech_prompts.py** : Templates prompts tech
- **agents/prompts/literature_prompts.py** : Templates prompts littérature

Phase 4 : API FastAPI

8. API Structure

- **api/app.py** : Application FastAPI principale
- **api/models/request_models.py** : Modèles Pydantic requêtes
- **api/models/response_models.py** : Modèles Pydantic réponses
- **api/routes/tech_agent_routes.py** : Routes agent tech
- **api/routes/literature_agent_routes.py** : Routes agent littéraire
- **api/services/agent_service.py** : Logique métier agents
- **api/middleware/auth_middleware.py** : Authentification



DATASETS À TRAITER (Emplacements Confirmés)

Tech Books CSV (dans rag/tech_rag/data/)

title,description,author,isbn10,isbn13,publish_date,edition,best_seller,top Rated,review_count,price

Literature Books (dans rag/literature_rag/data/)

Dataset 1: isbn13,title,authors,categories,description,published_year,average_rating...

Dataset 2: booktitle,author,rating,voted,score



FONCTIONNALITÉS CLÉS

Mode Chatbot Individuel

- Conversation 1-à-1 avec chaque agent
- Historique des conversations sauvegardé
- Personnalisation selon profil utilisateur

Cross-Recommendations

- Agent Tech → suggestions littéraires selon style de code
- Agent Littéraire → suggestions tech selon préférences lecture

Validation Croisée

- Agents se valident mutuellement sur faits/contexte

CONFIGURATION TECHNIQUE

Models Configuration

python

```
model_config = {  
    "tech_agent": {  
        "model_name": "google/flan-t5-base",  
        "max_length": 512,  
        "temperature": 0.7  
    },  
    "literature_agent": {  
        "model_name": "google/flan-t5-base",  
        "max_length": 512,  
        "temperature": 0.8  
    }  
}
```

RAG Configuration

python

```
rag_config = {  
    "embedding_model": "sentence-transformers/all-MiniLM-L6-v2",  
    "chunk_size": 512,  
    "top_k_results": 5,  
    "similarity_threshold": 0.7,  
    "tech_data_path": "rags/tech_rag/data/",  
    "literature_data_path": "rags/literature_rag/data/",  
    "tech_chroma_path": "rags/tech_rag/chroma_db/",  
    "literature_chroma_path": "rags/literature_rag/chroma_db/"  
}
```



POINTS CRITIQUES

1. **Performance** : Models flan-t5-base légers mais suffisants
2. **Mémoire** : ~2-3GB VRAM total pour les 2 agents
3. **Données** : CSV dans rags//data/, *embeddings en cache dans rags//chroma_db/*
4. **UX** : Interface chat fluide, réponses <3s



FICHIERS PRIORITAIRES À CRÉER (Ordre de Priorité)

1. Configuration Django de Base

- `config/settings/base.py` : Configuration Django + PostgreSQL
- `config/settings/development.py` : Settings développement
- `config/urls.py` : URLs racine avec inclusions apps
- `config/wsgi.py` et `config/asgi.py` : Configuration serveur

2. Applications Django

- `apps/accounts/models.py` : User, UserProfile
- `apps/books/models.py` : TechBook, LiteratureBook, Recommendation
- `apps/chat/models.py` : ConversationHistory, Message
- `apps/*/admin.py` : Interfaces d'administration

3. Scripts Import Données

- `scripts/import_tech_books.py` : Import CSV tech vers PostgreSQL + ChromaDB
- `scripts/import_literature_books.py` : Fusion + import CSV littérature
- `scripts/create_embeddings.py` : Génération embeddings depuis CSV

4. Gestionnaires RAG

- `rag/shared/embedding_manager.py` : Interface sentence-transformers
- `rag/tech_rag/tech_rag_manager.py` : RAG spécialisé tech
- `rag/literature_rag/literature_rag_manager.py` : RAG spécialisé littérature

5. Agents LLM

- `agents/base_agent.py` : Classe de base commune
- `agents/tech_agent.py` : Agent tech avec flan-t5-base + RAG tech
- `agents/literature_agent.py` : Agent littéraire avec flan-t5-base + RAG littérature
- `agents/prompts/tech_prompts.py` : Templates prompts tech
- `agents/prompts/literature_prompts.py` : Templates prompts littérature

6. API FastAPI

- `api/app.py` : Application FastAPI principale
- `api/routes/tech_agent_routes.py` : Endpoints agent tech
- `api/routes/literature_agent_routes.py` : Endpoints agent littéraire
- `api/services/agent_service.py` : Bridge API ↔ Agents

RÉSULTAT ATTENDU

Un système fonctionnel où :

1. **CSV importés** depuis `rag/*/data/` vers PostgreSQL + ChromaDB
2. **Utilisateur s'inscrit** via `apps/accounts/`
3. **Choisit son agent** via `apps/chat/`
4. **Pose une question** : "Je veux apprendre React" ou "J'ai aimé Murakami"
5. **Agent répond** via API FastAPI + RAG personnalisé
6. **Historique sauvegardé** dans `apps/chat/models`

COMMANDES DE DÉMARRAGE

bash

Setup environnement (déjà fait)

source venv/bin/activate

Django setup (à faire)

cd config/

python manage.py migrate

python manage.py createsuperuser

python manage.py runserver

Import données (à créer)

python scripts/import_tech_books.py

python scripts/import_literature_books.py

API setup (terminal séparé)

cd api/

uvicorn app:app --reload

ORDRE D'IMPLÉMENTATION RECOMMANDÉ

1. **Django setup** : config/settings/ + apps/*/models.py
 2. **Import scripts** : scripts/import_*.py pour charger tes CSV
 3. **RAG managers** : rags/*/rag_manager.py avec ChromaDB
 4. **Agents de base** : agents/*_agent.py avec flan-t5-base
 5. **API** : api/app.py + routes pour tester agents
 6. **Interface web** : apps/chat/ pour interface utilisateur
-

 **CLAUDE** : Commence par la configuration Django dans config/settings/, puis crée les modèles dans apps/. Les CSV sont déjà dans rags/*/data/, utilise-les pour alimenter le système RAG.

PERSONNALITÉS DES AGENTS

Agent Tech

- **Style** : Pragmatique, orienté solutions
- **Exemples** : "Pour React, je recommande 'Fullstack React' (4.6 ★, \$39) - projets complets, très à jour avec React 18"
- **Logique** : Adapte selon niveau utilisateur (débutant/expert)

Agent Littéraire

- **Style** : Cultivé, découvreur, empathique

- **Exemples** : "Ah, Camus ! Tu pourrais adorer 'La Nausée' de Sartre - cette exploration du vide existentiel..."
- **Logique** : Mix découvertes/zone de confort selon profil

TÂCHES PRIORITAIRES

Phase 1 : Fondations (URGENT)

1. Setup Django complet

- Configurer settings.py avec PostgreSQL
- Créer modèles : User, UserProfile, TechBook, LiteratureBook, Recommendation
- Migrations et admin interface

2. Système RAG fonctionnel

- Scripts import des CSV vers ChromaDB
- Génération embeddings avec sentence-transformers
- Tests de recherche vectorielle

3. Agents de base

- Classes TechAgent et LiteratureAgent avec flan-t5-base
- Prompts système pour chaque personnalité
- Integration RAG dans génération de réponses

Phase 2 : API et Interface

4. API FastAPI

- Endpoints pour chaque agent
- Modèles Pydantic pour requêtes/réponses
- Integration avec Django

5. Interface Django

- Pages d'authentification
- Chat interface pour les 2 agents
- Dashboard utilisateur

DATASETS À TRAITER

Tech Books CSV

title,description,author,isbn10,isbn13,publish_date,edition,best_seller,topRated,review_count,price

Literature Books (2 fichiers à fusionner)

Dataset 1: isbn13,title,authors,categories,description,published_year,average_rating...

Dataset 2: booktitle,author,rating,voted,score

FONCTIONNALITÉS CLÉS

Mode Chatbot Individuel

- Conversation 1-à-1 avec chaque agent
- Historique des conversations
- Personnalisation selon profil utilisateur

Cross-Recommendations

- Agent Tech → suggestions littéraires
- Agent Littéraire → suggestions tech selon style

Validation Croisée

- Agents se valident mutuellement sur faits/contexte

CONFIGURATION TECHNIQUE

Models Configuration

python

```
model_config = {  
    "tech_agent": {  
        "model_name": "google/flan-t5-base",  
        "max_length": 512,  
        "temperature": 0.7  
    },  
    "literature_agent": {  
        "model_name": "google/flan-t5-base",  
        "max_length": 512,  
        "temperature": 0.8  
    }  
}
```

RAG Configuration

python

```
rag_config = {  
    "embedding_model": "sentence-transformers/all-MiniLM-L6-v2",  
    "chunk_size": 512,  
    "top_k_results": 5,  
    "similarity_threshold": 0.7  
}
```



POINTS CRITIQUES

1. **Performance** : Models flan-t5-base légers mais suffisants
2. **Mémoire** : ~2-3GB VRAM total pour les 2 agents
3. **Données** : Protéger CSV originaux, embeddings en cache
4. **UX** : Interface chat fluide, réponses <3s



FICHIERS PRIORITAIRES À CRÉER

Agents

- `agents/base_agent.py` : Classe de base
- `agents/tech_agent.py` : Agent spécialisé tech
- `agents/literature_agent.py` : Agent littérature
- `agents/prompts/tech_prompts.py` : Templates prompts tech
- `agents/prompts/literature_prompts.py` : Templates prompts littérature

RAG

- `rags/tech_rag/tech_rag_manager.py` : Gestionnaire RAG tech
- `rags/literature_rag/literature_rag_manager.py` : Gestionnaire RAG littérature
- `rags/shared/embedding_manager.py` : Gestionnaire embeddings partagé

Scripts Utilitaires

- `scripts/import_tech_books.py` : Import CSV tech vers DB
- `scripts/import_literature_books.py` : Import + fusion CSV littérature
- `scripts/create_embeddings.py` : Génération embeddings

Django

- `web/apps/accounts/models.py` : Modèles utilisateur
- `web/apps/books/models.py` : Modèles livres

- `web/apps/chat/views.py` : Interface chat
- `web/config/settings/base.py` : Configuration Django

RÉSULTAT ATTENDU

Un système fonctionnel où :

1. **Utilisateur s'inscrit** et configure son profil
2. **Choisit son agent** (Tech ou Littéraire)
3. **Pose une question** : "Je veux apprendre React" ou "J'ai aimé Murakami"
4. **Agent répond** avec recommandations personnalisées et contexte enrichi
5. **Historique sauvegardé** pour amélioration continue

COMMANDES DE DÉMARRAGE

```
bash
```

```
# Setup environnement
```

```
source venv/bin/activate
```

```
pip install -r requirements.txt
```

```
# Django setup
```

```
cd web
```

```
python manage.py migrate
```

```
python manage.py createsuperuser
```

```
python manage.py runserver
```

```
# API setup (terminal séparé)
```

```
cd api
```

```
uvicorn app:app --reload
```

 **CLAUDE** : Commence par la Phase 1, crée les fichiers prioritaires un par un en respectant cette architecture. Focus sur un système fonctionnel minimal avant d'ajouter des features avancées.